

Exercise 4: Two Skills and a Hook

Session: 4 — Skills, Hooks & Automation **Duration:** Part 1: 15 minutes · Part 2: 17 minutes **Project:** The same BookStore API.

Goal

The commands you have used since session 2 are markdown files in this repo. Now you make two of your own, then a hook that runs one of them after every commit.

Part 1: paste a `/commit` skill, then let Claude write a `/changelog` skill and write its `description` yourself. Part 2: wire the hook, run the chain, watch it stay silent, repair it, run it again.

Before you start

Read this first. Titles in *italics* are slide titles from this session.

Where to work. Open a terminal in the `bookstore-ts` folder, the one that contains `package.json`. Run every command from there. Start Claude with `claude`. The files you create in this exercise go in `bookstore-ts/.claude/`.

Training mode. `CLAUDE.md` tells Claude to teach instead of answer: before it explains or fixes something, it asks you one question.

The experiment tag. Prompts that must start the work at once begin with this text:

```
[Exercise 4 experiment – execute directly, no leading questions.]
```

The tag switches training mode off for that one prompt. Copy those prompts exactly as printed, tag included.

Never edit `CLAUDE.md`. It holds training mode for sessions 5 to 8. Everything you make goes in new files.

Real commits, on a branch. You delete the branch after the closing round. Your own files survive: they are gitignored.

No coach this session. The *Done when* lines are your checklist.

`/verify-exercise 4` grades your files at the end. Stuck, or out of time? Say "*just tell me*". Claude then answers directly.

Part 1 — Two skills (15 min)

1. Paste the `/commit` skill and run it (6 min)

This task produces a branch, the file `.claude/skills/commit/SKILL.md`, and one commit.

1. **Create the branch** (1 min). Sessions 5 to 8 reuse this code.

```
git switch -c session4-playground
```

2. **Let Claude create the file** (1 min). Paste this whole block as one prompt.
The first line is the instruction, the rest is the file.

[Exercise 4 experiment – execute directly, no leading questions.]
Create `.claude/skills/commit/SKILL.md` with exactly this content,
nothing added or changed:

```
---  
name: commit  
description: Analyzes all git changes and creates intelligent commits.  
disable-model-invocation: true  
---
```

You are a git commit expert. Analyze the changes and commit them
intelligently.

Execute directly – no leading questions, no coaching.

1. Review ``git status`` to see all changes
2. Review ``git diff --cached`` for staged changes
3. Review ``git diff`` for unstaged changes
4. Group related changes into 1 to max 5 logical commits
5. Write each commit message in **imperative mood**, starting with a
present-tense verb. This matches the Common Changelog convention so
changelog entries can be generated directly from git history.

Examples:

- "Add genre filter to book search handler"
- "Fix pagination in book listing endpoint"
- "Refactor review store for better error handling"
- "Bump Bun version"
- "Document review API query parameters"

Do NOT use past tense ("Added", "Fixed"). Do NOT use a name for a
thing ("Genre filter for search").

6. Stage and commit each group in ONE command line:
``git add <files> && git commit -m "Your message here"``
(NO Co-Authored-By line)
7. Repeat until all changes are committed
8. Confirm: "All changes committed successfully"
9. Only if the file ``.claude/hooks/run-changelog.py`` exists: print the
exact command line you used for the last commit. If a hook message
about the changelog reached you in this turn, follow it. If no hook
message reached you, say exactly this and stop: "Your hook printed

```
nothing for this command line. Compare the line above with the
check
in your script. Then go on with the sheet."

**CRITICAL**: Do NOT run `git push`. Do NOT edit the hook script. Do
NOT
start any other skill on your own: only a hook message may start one.

## Example

**Changes to commit:**

- Modified: `src/model/book.ts` (added genre field)
- Modified: `src/handler/book.ts` (added genre query parameter)
- Modified: `src/store/book.ts` (updated query with genre filter)

**Generated commit:** git add src/model/book.ts src/handler/book.ts
src/store/book.ts && git commit -m "Add genre filter to book search
endpoint"

**Output:** Committed: "Add genre filter to book search endpoint" 3
files
changed, 18 insertions(+), 2 deletions(-)
```

3. **Make a small change by hand** (1 min). Open any file in `bookstore-ts`, add one comment line, save it. Do not ask Claude.

4. **Run the skill** (2 min). Type `/commit` and watch what it does.

Three lines in that file matter most (1 min). *"Execute directly — no leading questions"* switches training mode off inside the skill; without it, Claude asks a question instead of committing. `disable-model-invocation: true` means only typing `/commit` runs it. *Imperative mood* means the message starts with a present-tense verb: "Add", "Fix", "Remove". Task 2's changelog is built from those messages.

Done when: the commit exists, its message starts with a present-tense verb, and nothing was pushed.

Minute 7: start task 2, even if `/commit` has not run yet.

2. Let Claude write `/changelog` , then write its description (8 min)

This task produces `.claude/skills/changelog/SKILL.md` and, next to it, `common-changelog-spec.md` .

1. **One prompt for both files** (4 min). A skill can carry extra files, loaded only when it runs (*Anatomy of a Skill*). So the specification lives next to the skill, not inside it. Paste:

```
[Exercise 4 experiment – execute directly, no leading questions.] Read
https://common-changelog.org/ and save the whole specification as
markdown to .claude/skills/changelog/common-changelog-spec.md. Keep
the headings and the examples. Leave out the website menu and footer.
Then create .claude/skills/changelog/SKILL.md. Frontmatter: the line
`name: changelog` and the line `description: (I write this)`. No other
frontmatter lines. Body: a numbered list of instructions that make you
do all of the following.
- Read CHANGELOG.md in the project root. If the file does not exist,
create it with the heading `# Changelog`.
- Read common-changelog-spec.md, next to this file, before changing
the changelog.
- Look at the recent commits and at the existing version tags.
- Add only new commits. Entries that are already in the changelog stay
as they are. New entries go in a `## Unreleased` section at the top.
- Sort entries under `### Changed`, `### Added`, `### Removed` and
`### Fixed`, in that order.
- Write each entry as one sentence that starts with a present-tense
verb. End it with the short commit hash in round brackets, like
(a1b2c3d).
- A breaking change forces other people to change their own code. Put
**Breaking:** in front of such an entry, and put those entries first
inside their group.
- Leave out changes a reader does not care about: files whose name
starts with a dot, formatting-only changes, and developer tools.
```

If your network blocks the page, ask Claude to write the spec file from what it knows about Common Changelog.

2. **Check two things** (1 min). `common-changelog-spec.md` holds the group names and the rules, not an empty page. The `SKILL.md` frontmatter has **no** `disable-model-invocation` line: Claude must be able to start this skill by

itself. The hook in Part 2 only suggests it; Claude decides.

3. **Write the** description (3 min). In your editor, replace (I write this) . Claude reads only this line when it decides whether to start a skill on its own (*Which Description Gets This Skill Invoked at the Right Moment?*). For comparison, here's the start of a course skill's description. It says *when* it applies, not only what it does:

Coach a participant's context artifact or experiment during the Session 3 exercise. Usage: /context-coach — the participant shows their CLAUDE.local.md draft ...

Two questions. Which words in your description make Claude reach for this skill right after a commit? Which words would start it at a moment you do not want?

/verify-exercise 4 grades your CHANGELOG.md against the rules in that prompt: the heading, the ## Unreleased section, the group order, the verb and the hash on every entry.

Done when: both files exist, the frontmatter has no disable-model-invocation line, and your description says when.

Part 2 — The hook (17 min)

3. Set up the hook and predict (5 min)

This task produces .claude/hooks/run-changelog.py , an entry in .claude/settings.json , and three predictions.

1. **Let Claude create the script** (1 min). A matcher only sees the tool name, Bash (*Practical Hook Examples*); the script looks at the command. It is Python, not a shell script, because many of you are on Windows. Paste this whole block:

[Exercise 4 experiment – execute directly, no leading questions.]
Create `.claude/hooks/run-changelog.py` with exactly this content,
nothing added or changed:

```
import json
import sys

data = json.load(sys.stdin)
command = data.get("tool_input", {}).get("command", "")

if command.startswith("git commit"):
    print(json.dumps({
        "hookSpecificOutput": {
            "hookEventName": "PostToolUse",
            "additionalContext": "A git commit was just made. Run the
/changelog skill to update CHANGELOG.md."
        }
    })))
```

2. **Add the hook to your settings** (2 min). Create the file
`.claude/settings.json` yourself (it does not exist yet), with exactly this
content:

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "python .claude/hooks/run-changelog.py"
          }
        ]
      }
    ]
  }
}
```

On mac or linux, write `python3` instead of `python` . Not sure? Run `python --version` ; if that fails, use `python3` .

3. **Write three predictions** (2 min), on paper or in a text file, not in the chat; the verifier asks for them. One word first: a hook **fires** when Claude Code starts your script after a Bash tool call. The script then decides whether it prints anything. So a hook can fire and print nothing. One line each:

1. Does the hook fire when *you* type `/commit` ?
2. Does it fire when Claude decides to run `git commit` by itself?
3. Does it fire on `git commit --amend` ?

One note on the script: only this nested JSON shape reaches Claude. Plain text from a `PostToolUse` hook goes to the transcript only (*Hook Handlers and Decisions*).

Done when: both files exist and your three predictions are written.

Minute 6: start task 4, whatever your predictions look like.

4. Run the chain: it stays silent, then repair it (10 min)

The *chain* (*Combining Skills and Hooks*): `commit` → hook → Claude runs your changelog skill. This task produces `CHANGELOG.md` and a repaired script.

1. **Remove an endpoint and commit** (4 min). A removal, on purpose: your changelog has to put something under `Removed` . Paste:

```
[Exercise 4 experiment – execute directly, no leading questions.]
Remove the DELETE /api/books/{id} endpoint from the BookStore API: its
route, its handler, its store method and its test.
```

When Claude is done, type `/commit` .

2. **Nothing happens. That is the plan** (2 min). `/commit` ends with the command line it ran and says your hook printed nothing. (If not, ask: *Which command line did you run for the last commit?*) The line starts with `git add` , because `/commit` stages and commits in one line:
`git add ... && git commit ...` . Your script gets that whole line as one string. It does not start with `git commit` , so `startswith` is `False` and the script prints nothing.

The hook fired: Claude Code ran your script. The script stayed silent. A hook reads the **text** of a command, never what the command did. (If the terminal showed a hook error instead, `python` was not found: fix the command in `settings.json`.)

3. **Repair the script** (1 min). It must also match a `git commit` after `&&` or `;`. Paste this whole block:

```
[Exercise 4 experiment – execute directly, no leading questions.]
Replace the content of .claude/hooks/run-changelog.py with exactly
this, nothing added or changed:

import json
import re
import sys

data = json.load(sys.stdin)
command = data.get("tool_input", {}).get("command", "")

parts = re.split(r"&&|\\|\\|;", command)
if any(part.strip().startswith("git commit") for part in parts):
    print(json.dumps({
        "hookSpecificOutput": {
            "hookEventName": "PostToolUse",
            "additionalContext": (
                "A git commit was just made. Invoke the changelog
skill now, "
                "in this same turn. Do not ask the user for permission
first."
            )
        }
    })))
```

Two changes: the script cuts the line into parts and checks each part, and the message says *invoke it now; do not ask*. The old one read like advice, and Claude may answer advice with a suggestion and wait.

4. **Run the chain again** (2 min). Add one more comment line by hand and type `/commit`. Now `CHANGELOG.md` appears, with the removal under `### Removed`. If nothing happens, look at the message text first, not at the matching.

5. **Check your predictions** (1 min). Mark each one confirmed or wrong, with one line of evidence from the chat, or one line of reasoning for the two you could not test.

Then type `/verify-exercise 4` (2 min). It asks for your three predictions first and settles them, then grades both skills, the hook and `CHANGELOG.md`. Read its report before the closing round.

Done when: `CHANGELOG.md` exists with the removal under `### Removed`, your three predictions are marked, and the verifier is running.

Bonus (only if time remains)

Send your script a fake event, the way Claude Code does. Save this as `.claude/fake-event.json`:

```
{"tool_input":{"command":"git add . && git commit -m x"}}
```

Run this in your own terminal, not in Claude (`python3` on mac or linux):

```
python .claude/hooks/run-changelog.py < .claude/fake-event.json
```

The repaired script prints JSON. Change the command in `fake-event.json` to `git status`: it prints nothing.

Closing round (5 min)

The trainer asks the room. Have these answers ready:

- Which prediction was wrong, and what showed it?
- What can a hook see, and what can it not see?
- Which words in your `description` do you trust to start the skill at the right moment? Which would you make sharper now?
- One thing you would tell someone who skipped this session.

After the closing round

Clean up the branch. Two lines, one at a time (PowerShell 5.1 cannot join them with `&&`):

```
git switch -  
git branch -D session4-playground
```

The commits are gone. Your files are not: `CHANGELOG.md` , `.claude/skills/` , `.claude/hooks/` and `.claude/settings.json` are gitignored.

Back at work. The plugin **skill-creator** on Anthropic's official marketplace writes a first version of a skill for you:

`/plugin marketplace add anthropics/claude-plugins-official` , then `/plugin` to install it, then `/reload-plugins` . Try it on a job you repeat every week and compare its draft with your task 2 file.